```
const ChildB: React.FC<DataProps> = ({ data }) => {
  return (
    <div>
      <h3>Child B Component</h3>
      <ChildC data={data} /> #C
    </div>
  );
};
const ChildC: React.FC<DataProps> = ({ data }) => {
  return (
    <div>
      <h4>Child C Component</h4>
      <p>{data}</p> #D
    </div>
  );
};
```

This example is located in `use-cases/PropDrillingExample.tsx`, and can be executed with the command: `VITE_NAME=PropDrillingExample npm run dev`.

The Context API offers a cleaner and more scalable solution. Instead of manually passing props through every layer, a context is created to hold the shared data, and the component tree is wrapped in a provider. Any component in the tree can then access the data directly using the `useContext` hook—bypassing intermediate layers entirely.

In listing 4.3, `DataContext` is created and provided with a value in `ParentComponent`. `ChildA` and `ChildB` no longer need to receive or forward props, while `ChildC` consumes the data directly from context. This refactor simplifies component interfaces and reduces boilerplate, making the codebase easier to maintain as applications scale.

Listing 4.3 Context API solution

```
const DataContext = createContext<DataProps>({ data: '' }); #A
const ParentComponent: React.FC = () => {
  const data = 'Hello from Parent!';
  return (
    <DataContext.Provider value={{ data }}>
      <div>
        <h1>Parent Component</h1>
        <ChildA />
```